# 🚨 GRADE BOOSTER: ALERTS & SPECIFICATIONS

*The "Must-Know" section for exams and projects based on Dr. Sanan's specific emphasis.*

## 💀 Exam Traps & "Distinction" Nuances

- **The "What is an OS?" Trap:** Do not define an OS simply by the hardware it runs on.
  - *The Trap:* Thinking a **Raspberry Pi** is an OS (it's hardware). Thinking **libc** (C library) is an OS (it's a library). Thinking a **GUI** is an OS (it's just an interface program).
  - *The Correct View:* The OS is the software layer managing hardware/resources (kernel + system programs). Firmware on a Samsung phone *is* an OS; the software on a drone *is* an OS.
- **Monolithic vs. Microkernel (The EV Example):**
  - *The Trap:* Assuming Linux is always the best choice because it's popular.
  - *The Insight:* **Linux** is Monolithic (fast, but one crash kills all). **EVs (Electric Vehicles like Tesla/NIO)** use **Microkernels** (like QNX or HarmonyOS).
  - *Why?* Reliability. If the infotainment crashes in a Monolithic kernel, the brakes might stop working. In a Microkernel, only the music stops. **Expect a question on reliability vs. performance tradeoffs here.**
- **AI Policy (The "Fail" Condition):**
  - *The Rule:* You *can* use ChatGPT for learning/practice. You **CANNOT** use it during the Graded Quizzes/Assessments.
  - *The Consequence:* If caught using AI during an assessment, you fail the **entire module**, not just the lab.
- **Reading vs. Watching:**
  - *Prof's Hint:* "Industry prefers SIT students... but feedback says reading comprehension could improve." Read the documentation/books *before* watching videos. This is a skill being tested.

## 🛠 Project & Assignment Intel

- **Assessment Structure:**
  - **Labs (20%):** 4 submissions (5% each).
  - **Quizzes (80%):** 4 Quizzes (20% each).
  - **Quiz Format:** Quizzes are **not** just MCQ. They include a **practical coding component** where you must solve a problem using the xv6 environment.
- **Environment Setup:**
  - **Required:** WSL2 (Windows), Ubuntu VM, or macOS.
  - **Critical Tools:** QEMU (Emulator) and RISC-V GCC toolchain.

    - *Tip:* If WSL gives you trouble, switch to an Ubuntu VM immediately to avoid wasting time.
- **xv6 Context:**
    - We use **RISC-V** architecture (not x86).
    - We use **QEMU** because virtualizing a custom OS kernel is faster and easier for debugging (using GDB) than VirtualBox/VMware, which are GUI-heavy.

---

# 📖 LECTURE RECONSTRUCTION

## 1. Module Philosophy & Introduction

**Concept:** The shift from "How to *use* an OS" to "How to *build* an OS."
**The Lecture Deep-Dive:**
The module uses **xv6**, a re-implementation of Unix v6 by MIT.

- **Why xv6?** Linux is massive (millions of lines). xv6 is tiny (~10,000 lines). It allows you to see the *entire* lifecycle of a system call or memory page.
- **The Nokia/Blackberry Parable:** Why did iOS/Android kill Nokia?
    - *It wasn't the touchscreen.* (Nokia had them).
    - *It was the API/Ecosystem.* Nokia required a PhD to program apps for. Apple/Google created developer-friendly APIs. An OS is only as good as the applications it enables developers to build easily.

## 2. Operating System Definitions & Goals

**Concept:** An intermediary between User/Applications and Hardware.
**Three Main Goals:**

1. **Abstraction:** Hiding messy hardware details (e.g., "Write to file" vs. "Move disk head to sector 400").
2. **Resource Allocation:** Deciding who gets the CPU/Memory (Solving contention: "I want to play a game, but a critical system update is running").
3. **Protection/Control:** preventing a user from crashing the system or reading another user's memory.
    - *Historical Horror Story:* Old OSs allowed direct hard drive access. A bad command could physically drive the needle off the platter, breaking the disk. Modern OSs prevent this via drivers.

## 3. OS Architecture Models

**Technical Comparison:**

| Architecture | Structure | Pros | Cons | Examples |
|---|---|---|---|---|
| **Layered** | Rings (Hardware -> Layer 1 -> ... -> User) | Easy to debug (test layer by layer). | Hard to define layers; slow communication (traversing layers). | Historic concepts. |
| **Modular** | Core + Loadable Modules (OO approach). | Extensible (add drivers without reboot). | Complex dependencies between modules. | Linux, Solaris. |
| **Monolithic** | **Entire OS runs in Kernel Space.** | **Performance.** Direct function calls are fast. | **Reliability.** One buggy driver crashes the whole system (Blue Screen of Death). | **Linux, xv6**, Unix. |
| **Microkernel** | Tiny Kernel (IPC, Scheduling) + User Space Services. | **Reliability & Security.** If a service crashes, restart it. Kernel stays up. | **Performance Overhead.** Message passing between services is slower than function calls. | **QNX, HarmonyOS, Mach** (macOS core). |

## 4. Introduction to xv6 & RISC-V

**Concept:** A simplified Unix-like OS running on RISC-V (Open Source Instruction Set Architecture).
**The xv6 Structure:**

- **Monolithic Kernel:** Kernel and User space are distinct.
- **Memory:** Shared memory space for the kernel (simplifies design but risky in real life).
- **Workflow:**
  1. `make qemu`: Compiles kernel + user programs.
  2. `mkfs`: Creates a file system image.
  3. **Boot:** BIOS -> Bootloader -> Kernel Init -> `main()` -> `userinit()` (creates first process).
- **Mode Switch:**
  - **User Mode:** Restricted. Cannot touch hardware directly.
  - **Kernel Mode:** God mode. Triggered by **Traps** (System Calls or Interrupts).
  - *Mechanism:* Save context (registers) -> Switch mode -> Execute handler -> Restore context.

## 5. System Call Demos (The "Code Logic")

*The prof walked through specific demos. These are likely the basis for Lab 1.*

## Demo 1: I/O ( `read` / `write` )

Unix treats I/O as file descriptors. `0` = stdin, `1` = stdout.

```
// Logic: Read from input, write to output.
char buf[64];
while(1) {
    // read(fd, buffer, size)
    // Returns 0 on EOF (Ctrl+D), >0 bytes read, -1 on error
    int n = read(0, buf, sizeof(buf));
    if (n <= 0) break;
    write(1, buf, n); // Write exactly 'n' bytes to stdout
}
```

## Demo 2: File Creation ( `open` )

Need specific flags to create/write files.

```
// O_CREATE: Make if missing. O_TRUNC: Wipe if exists. O_WRONLY: Write only.
int fd = open("out", O_CREATE | O_WRONLY | O_TRUNC);
printf("open returned fd %d\n", fd); // Likely returns 3 (since 0,1,2 are
taken)
write(fd, "SIT\n", 4);
```

## Demo 3: Process Creation ( `fork` )

The core multitasking primitive.

```
int pid = fork();
if (pid == 0) {
    // Child Process executes this
    printf("I am the child\n");
} else {
    // Parent Process executes this
    // 'pid' holds the Child's ID here
    printf("I am the parent of %d\n", pid);
}
```

## Demo 4: Process Replacement ( `exec` )

Replaces the current memory with a new program (e.g., `echo` ).

```
char *argv[] = {"echo", "hello", 0};
exec("echo", argv);
// Code below exec ONLY runs if exec fails
printf("Exec failed!\n");
```

### Demo 7/8: Inter-process Communication ( `pipe` )

Pipes are buffers maintained by the kernel.

```
int fds[2];
pipe(fds); // fds[0] is read-end, fds[1] is write-end

if (fork() == 0) {
    // Child: Write to pipe
    write(fds[1], "Hello", 5);
} else {
    // Parent: Read from pipe
    char buf[100];
    read(fds[0], buf, 5);
}
```

---

## 🗣 Q&A CORNER

- **Q: Can we use Kali Linux / Other Distros?**
  - **Prof A:** Yes, but **Ubuntu** is recommended because it's the standard tested environment. If you use Kali, you need to ensure you have the correct C libraries and RISC-V toolchain manually.
- **Q: Is the quiz Open Book?**
  - **Prof A:** *Ambiguous in transcript, but leans towards "Restricted Resources".* There is no lockdown browser, so you have access to your local files, but **AI is strictly banned**. You are expected to code solutions, not copy-paste.
- **Q: Why use QEMU instead of VirtualBox?**
  - **Prof A:** VirtualBox/VMware are GUI-oriented. QEMU allows us to emulate the CPU architecture (RISC-V) specifically and integrates better with `make` and `gdb` for kernel debugging via the command line.

---

## ⚡ EXECUTIVE SUMMARY (TL;DR)

1. **Safety First:** Do not use ChatGPT during graded labs/quizzes. Instant fail on the module if caught.
2. **OS Definition:** It's the software managing hardware complexity and resource allocation. It makes hardware usable for developers (API importance).
3. **Architecture: Monolithic** (xv6/Linux) = Fast but fragile. **Microkernel** (EVs) = Reliable but slower.
4. **xv6:** A 10k-line educational OS based on Unix v6, running on RISC-V. It is a monolithic

kernel.

5. **Coding Core:** Master `fork` (create process), `exec` (run program), `pipe` (communicate), and file descriptors ( `0` , `1` , `open` )—this is the foundation of Lab 1.